

Plane on AWS — Deployment Notes

A record of what we're planning for the AWS deployment and why. Design reasoning only — implementation detail lives in the per-change docs alongside this one.

Status: design. The only piece built so far is the SQS broker switch — see [sqs-broker.md](#). Nothing is deployed to AWS yet.

1. What this is

A fork of **Plane v1.3.1** — open-source project management (issues, cycles, modules, pages, analytics) — customised for OneByZero: branding, an Azure AD / Entra ID login option, env-based admin seeding, and telemetry removed.

Today it can only be deployed the way upstream ships it — docker-compose on one machine, a Docker Swarm script, an all-in-one container, or upstream's public Helm chart. **There is no AWS setup in the repo at all.**

2. How it runs today

`docker-compose.yml` starts 13 containers.

Container	What it is
-----------	------------

proxy	Caddy — handles HTTPS and routes every URL
web	Main app. Pre-built files served by nginx
admin	"God mode" instance admin. Also pre-built files
space	Public shareable pages. A real Node process that renders pages
live	Collaborative editing. Holds open connections to browsers
api	Django — the brain. The only thing that owns the data
worker	Does slow jobs in the background (email, exports, webhooks)
beat-worker	Alarm clock. Triggers scheduled jobs
migrator	Sets up the database once, then exits
plane-db	PostgreSQL
plane-redis	Redis — cache + messaging
plane-mq	RabbitMQ — the job queue
plane-minio	MinIO — file storage

How a request is routed today — Caddy owns the whole table:

```

/api/*  /auth/*  /static/*  ->  api
/spaces/*                ->  space
/god-mode/*              ->  admin
/live/*                  ->  live
/uploads/*                ->  minio          <- file upl
/*  (everything else)    ->  web

```

Who talks to what:

```

api    ->  database, redis, queue, file storage

```

```
worker -> database, file storage      (woken by the qu
beat   -> database (its schedule), queue (publishes jobs
live   -> redis, and calls api to check logins
space  -> calls api while rendering pages
```

3. What we're planning

Routing moves from Caddy to an AWS load balancer, and the /uploads/* route disappears because files will go straight to S3:

```
/api/*   /auth/*   /static/*   ->  api
/spaces/*                ->  space
/god-mode/*              ->  admin
/live/*                  ->  live
/*   (everything else)   ->  web

browser -> S3 directly      ->  file uploads (never touc
```

```
Route 53  ->  ACM certificate  ->  Load Balancer
                                     |
                                     front-facing:  web, admin, space,
                                     no front door:  worker, beat, boot
                                     support:         Redis (no disk)
                                     managed:         RDS PostgreSQL, S3
```

4. Side by side

	Today	On AWS	Why
--	-------	--------	-----

HTTPS + routing	Caddy container, Let's Encrypt	Load balancer + ACM + Route 53	AWS-managed, no certificate renewal to babysit
Database	postgres container	RDS PostgreSQL	The only data we can't rebuild. Backups and patching handled for us
File storage	MinIO container	S3	The app already knows how to use real S3. Removes a container and a disk
File uploads	browser → Caddy → MinIO	browser → S3 directly	Uploads stop passing through our servers entirely
Cache / messaging	redis container + disk	container, no disk	Nothing in it is worth keeping across a restart
Job queue	rabbitmq container + disk	SQS — <i>implemented, see sqs-broker.md</i>	Removes a container, its disk, and gives us monitoring for free
Running the containers	docker-compose on one machine	ECS Fargate (or EKS)	No servers to patch or size
Database setup	migrator container	run-once task	Must finish before anything else starts

Config / secrets	<code>.env</code> files, some committed to git	Secrets Manager	Committed secrets must not reach AWS
Logs	files inside the container	CloudWatch Logs	Files inside a container are unreachable and fill the disk
Images	Docker Hub, upstream's account	ECR	Our own images, private
How it's defined	<code>docker-compose.yml</code>	CloudFormation	Written down, reviewable, repeatable
Number of images	7	5	proxy and minio disappear entirely

5. Decisions and reasoning

The job queue: SQS instead of RabbitMQ

Decided and implemented. Implementation notes, configuration and local setup are in [sqs-broker.md](#); the reasoning is below.

SQS is AWS's managed queue. Same job as RabbitMQ — hold job slips until a worker picks them up — but there's nothing to run, nothing to patch, and no disk to manage.

	RabbitMQ container	SQS
Container to run	yes	none

Disk to manage	yes	none
Durability	our problem	AWS's
Queue-depth monitoring	needs setting up	built in

That last row matters more than it looks. A dead worker is invisible — nothing errors, emails just quietly stop arriving. SQS gives us the queue-depth measurement for free, which is exactly the alarm this app needs.

The trade-offs:

- **Jobs can occasionally run twice.** SQS promises "at least once" delivery, not "exactly once". In practice that means a rare duplicate email. It can be reduced but not eliminated.
- **Some Celery monitoring tools stop working** — the ones that rely on RabbitMQ's protocol.
- **Local development would differ from AWS**, since docker-compose would still use RabbitMQ.
- It's a change to *how the app works*, not just where it runs — so it's a second variable if something goes wrong during the first deploy.

Amazon MQ is not recommended. It's AWS's managed RabbitMQ, but a single instance carries no availability guarantee, and a properly redundant one is considerably more expensive than SQS. If we're going to use a managed queue, SQS is the better option.

ECS Fargate vs EKS + Helm — still open, leaning ECS

	ECS Fargate	EKS + Helm
Ongoing charge just to exist	none	a per-cluster fee
Who patches the machines	AWS	us
Kubernetes version upgrades	not applicable	ours, roughly yearly, forced

Tools needed	CloudFormation only	CloudFormation and Helm
Head start available	none	upstream's Helm chart exists

This can be decided late. The network, the database, and all the app-side preparation in section 6 are identical either way.

Data layer: managed where it matters

- **RDS and S3 managed** — Postgres is the one thing we genuinely cannot rebuild, and S3 removes a whole container while the app already supports it
- **Redis stays a container with no disk** — it only holds cached copies and messages in transit. Losing it costs a minute of slower responses, nothing more
- **Queue moves to SQS** (above)

The load balancer replaces Caddy

It takes over the route table Caddy owns today. Two details we mustn't forget:

- **The idle timeout has to be raised.** The default would cut off collaborative editing sessions after a minute
- **Two redirect rules are needed** for the bare `/spaces` and `/god-mode` URLs — Caddy does these today, and without them both apps load broken

Merging containers — what we considered

Merge	Verdict
web + admin	Possible and easy. Both are just nginx serving files. Skipped for now — the saving is marginal, and it makes pulling future updates from upstream harder

worker + beat	Never. Scaling up workers would create multiple alarm clocks, and every scheduled job would fire twice
space + live	Technically the easiest (both are Node), operationally the worst. <code>live</code> holds unsaved edits in memory, so merging would make <code>space</code> un-restartable too
anything + api	Different language. Co-location at best, not a real merge
everything	That's the existing all-in-one image. Works, but can never run more than one copy

The rule we used: merge only if both halves die together, grow together, and neither holds anything precious.

6. What the deployment has to account for

Characteristics of this app that shape the design, regardless of which platform we pick.

The front-end images are tied to their web address. The domain is compiled into the JavaScript when the image is built, so an image built for a test environment physically cannot serve production. This breaks the usual "build once, test, then promote the same thing" approach — each environment needs its own build.

Database setup has to finish before anything else starts. The api, worker and beat all wait for it. They don't fail loudly if it hasn't run — they just hang indefinitely, which is a confusing way to discover the problem.

The alarm clock can only ever have one copy. Its schedule lives in the database, so two copies means every scheduled job fires twice — duplicate emails, things deleted twice. Any autoscaling has to exclude it.

Three services have no front door at all — worker, beat, and the setup job. Nothing routes to them. If they fail, nothing looks broken; things just quietly stop happening. That's why the queue-depth alarm

matters.

live is the only service holding anything in memory. Everything else can be killed and replaced at any moment with no consequence. Restarting `live` drops people's editing sessions, so it needs treating differently on deploys.

Uploads need the storage bucket to permit browser access directly. Since the browser uploads straight to S3, the bucket has to allow it. If we miss this, uploads fail silently in the browser while the app reports success.

7. Still to decide

Question	Options	Note
ECS or EKS	Fargate / EKS+Helm	Can be decided late — everything else is shared
Scoped SQS permissions	account-wide <code>ListQueues</code> / <code>kombu predefined_queues</code>	Today's policy needs account-wide <code>sqs:ListQueues</code> , which can't be narrowed to one queue. Worth resolving before production
Domain layout	one domain with paths (<code>/god-mode</code> , <code>/spaces</code>) or a subdomain per app	One domain is what the app is built for. Subdomains need a wildcard certificate and shared-cookie setup, and it's the least-tested path in this codebase
AWS account	new / existing	Also whether a deployment role already exists